



Six Engineering Projects That See Everything Coming

Night vision, a heads-up display, a sentry tower, an RF station, a mesh network, and a command map that fuses them all – six builds that stack into a personal situational-awareness system, the same architecture the new defense-tech companies are built on. Sense, connect, display, decide. Every one a sensor, a network, or a screen.

01	Digital Night-Vision Monocular	See in total darkness with a camera that watches infrared – and an illuminator nobody else can see	\$60
02	AR Heads-Up Display Goggles	A see-through display that floats heading, waypoints, and teammate positions in front of your eye	\$70
03	Autonomous Sentry Tower	A tower that spots motion, slews to it, and tracks it across the yard – a perimeter that watches on its own	\$90
04	RF Ground Station	A fixed station that watches the whole radio spectrum, decodes what's public, and flags anything new	\$80
05	Mesh Field Radio	Pocket radios that form their own network – text and position sharing with no cell towers at all	\$70
06	Command Map (Common Operating Picture)	One map that fuses every sensor you built – tracks, alerts, and teammates in a single live picture	\$25

SENSORS 06 · TRACKS 03 · ALERTS 01 · MESH 3 NODES · RX ONLY

● By Enes · @1nska · Electrical Engineer – a Build List field guide



[PROJECT 01]

Digital Night-Vision Monocular

See in total darkness with a camera that watches infrared – and an illuminator nobody else can see



\$60

beginner

1 weekend

// why this matters

Night vision is the original force multiplier: the side that can see in the dark owns the night. Real image-intensifier tubes cost thousands, but a digital version is astonishingly cheap – a camera with its infrared filter removed sees near-IR light your eyes cannot, and an IR illuminator floods the scene with light that is invisible to everyone but your sensor. Build it into a monocular with a small display and you can walk through a pitch-black room, read a sign at 30 metres, and watch wildlife that has no idea it is lit. It is a beginner build with a jaw-dropping demo, and it teaches the sensor and optics fundamentals behind every EO/IR system.

WHAT THIS PROVES TO A RECRUITER

You understand silicon sensor spectral response (why cameras see near-IR), active versus passive illumination, IR illuminator design and safety, low-light imaging trade-offs (gain, exposure, noise), and packaging optics – the foundations of electro-optical engineering.

→ **the job this maps to:** electro-optical (EO/IR) and imaging-systems engineering – night-vision, surveillance-camera, and imaging-sensor teams.

! YOUR OWN DARK, NOT SOMEONE ELSE'S

Night vision lets you see people who can't see you. Use it on your own property, on hikes, for wildlife, for astronomy – never to watch people who don't know you're there. And keep illuminators to low-power IR LEDs: high-power IR lasers or arrays can injure eyes precisely because you can't see them.

// how it works

- ▶ **Silicon sees infrared:** A camera's silicon sensor is sensitive well past visible red into near-infrared (about 700–1000 nm); manufacturers add an IR-cut filter so daylight colours look right. Remove it (a “NoIR” camera) and the sensor sees IR.
- ▶ **Active illumination:** In true darkness there is little IR to see, so you add your own: an array of 850 nm IR LEDs floods the scene with light invisible to human eyes (a faint red glow at most) but bright to the sensor. That is active night vision.
- ▶ **Passive vs active:** Image intensifiers amplify faint ambient light (passive, no emission); thermal cameras see heat. Digital IR-illuminated night vision is active – cheap and effective, but the illuminator can be seen by other IR cameras.
- ▶ **Low-light imaging:** Higher sensor gain and longer exposure pull more from the dark at the cost of noise and motion blur; a fast lens gathers more light. Balancing gain, exposure, and illuminator power is the core tuning.
- ▶ **The monocular:** A small display behind an eyepiece lens shows the live IR video, and everything mounts in a tube with the camera and illuminator forward – a self-contained device you hold to your eye.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
NoIR camera + compute	Raspberry Pi Zero 2 + NoIR camera	1	\$30
IR illuminator	850 nm IR LED board	1	\$6
Display	2–2.8" TFT / OLED	1	\$12
Eyepiece + tube	eyepiece lens, 3D-printed housing	1	\$8
Battery	LiPo + boost	1	\$4

// build steps

1 Get IR video

Connect the NoIR camera to the Pi and view video. Point a TV remote at it and see its LED flash bright – proof the sensor sees IR.

2 Add the illuminator

Mount the 850 nm LED board next to the lens and turn the lights off. Watch the room appear on screen while it stays pitch black to your eyes.

3 Tune for the dark

Adjust gain, exposure, and frame rate for the best image in true darkness. Note the trade-off between brightness, noise, and blur.

4 Build the monocular

Mount camera, illuminator, display, eyepiece lens, and battery in a printed tube so you can hold it to your eye. Add a power switch and an illuminator dimmer.

5 Field test

Take it outside at night. Read a sign at distance, find the range limit of your illuminator, and watch wildlife. Film the demo – then explain why the illuminator is visible to other cameras.

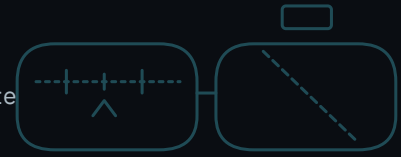
// interview questions

- Why can a silicon camera sensor see near-infrared light?
- What is the difference between active and passive night vision?
- Why do cameras include an IR-cut filter by default?
- How do gain, exposure, and lens aperture trade off in low light?
- What are the eye-safety concerns with IR illuminators?
- How does an image-intensifier tube differ from a digital IR camera?

[PROJECT 02]

AR Heads-Up Display Goggles

A see-through display that floats heading, waypoints, and teammate positions in front of your eye



\$70

advanced

2 weekends

// why this matters

The most talked-about piece of modern soldier equipment is the augmented-reality headset: a see-through display that overlays a compass, waypoints, teammate locations, and sensor alerts onto the real world, so the wearer never looks down at a map. You can build the core of one. A small bright display reflected off a half-mirror in front of your eye gives you a see-through HUD; an IMU compass and a GPS feed it heading and position; a radio brings in teammate positions and alerts. Turn your head and the bearing to the waypoint slides across your view. It is optics, embedded systems, and UI design in one build, and the first-person footage is unlike anything else in this series.

WHAT THIS PROVES TO A RECRUITER

You understand optical see-through display design (beam splitters, collimation, eye relief), heading and position sensing (IMU + magnetometer + GPS), rendering a stable HUD from sensor data, and designing information for a glanceable display – the core of AR/HUD and wearable-systems engineering.

→ **the job this maps to:** AR / HUD, wearable-systems, and human-machine-interface engineering – defense-headset, avionics-HUD, and AR-hardware teams.

! EYES FIRST

A display inches from your eye must be dim enough and properly focused through a collimating lens, and it must never be worn while driving, cycling, or in traffic. Keep most of the field of view clear: a HUD is an aid, not a replacement for looking.

// how it works

- ▶ **The optical combiner:** A small bright display is reflected off a half-silvered mirror (a beam splitter) angled in front of the eye. The mirror passes the real scene and reflects the display, so the graphics appear overlaid on the world.
- ▶ **Focus and eye relief:** A collimating lens between the display and the combiner sets the image's virtual distance so it is sharp alongside distant objects. Eye relief and combiner size set your usable field of view.
- ▶ **Heading and position:** An IMU with a magnetometer gives head heading (fused for stability); a GPS module gives position. From these you compute the bearing and distance to any waypoint or teammate.
- ▶ **Rendering the HUD:** A microcontroller draws a compass tape, a waypoint marker at its relative bearing, distance, and alert icons – high-contrast, minimal graphics that read at a glance without cluttering the view.
- ▶ **Bringing in the team:** A radio (your mesh node from project 5, or Bluetooth from a phone) supplies teammate positions and sensor alerts from the sentry or RF station, so the HUD shows the whole picture.

// key design decisions

- **Why a half-mirror combiner instead of a screen you look at?** A HUD only works if the information sits on the world – you must see through it. A beam splitter reflects the display into your eye while passing the real scene, so the overlay floats in front of you. A solid screen would block the view, which defeats the purpose.
- **Why does the display need a collimating lens?** Your eye cannot focus on a screen two inches away. A lens between display and combiner places the image at a comfortable virtual distance, near infinity, so the overlay and the distant world are in focus together. Without it the HUD is a blur.
- **Why fuse heading instead of using the magnetometer alone?** A magnetometer alone is noisy and lags, so a compass overlay would jitter and swim as you turn your head. Fusing the gyro (fast, but drifts) with the magnetometer (slow, but absolute) yields a smooth, stable heading – essential for a HUD that has to feel locked to the world.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Micro display	0.96–1.3" bright OLED, or small TFT	1	\$10
Beam splitter + lens	50/50 half-mirror glass, collimating lens	1	\$18
IMU + magnetometer	BN0055 / ICM-20948	1	\$14
GPS module	NEO-6M / 8M	1	\$12
ESP32 + battery + frame	MCU, LiPo, 3D-printed goggle mount	1	\$16

// build steps

1 Prove the optics

Mount the display, lens, and half-mirror on a bench and look through: adjust distances until the display image is sharp while you focus on the far wall. This is the hardest step – nail it first.

2 Draw a compass

Fuse IMU and magnetometer for heading and render a compass tape on the display. Turn your head and confirm the tape scrolls smoothly and settles on north.

3 Add waypoints

Read GPS, define a waypoint, and draw a marker at its relative bearing with distance. Walk toward it and watch the marker move and the distance drop.

4 Build the goggle

Mount everything on a 3D-printed frame or a pair of safety glasses, with the combiner in front of one eye and the battery and MCU on the strap. Balance and secure it.

5 Connect the picture

Feed teammate positions and alerts in over radio and render them. Film first-person through the combiner – the shot that shows what a HUD really is.

// interview questions

- How does an optical see-through HUD overlay graphics on the real world?
- Why is a collimating lens required in a near-eye display?
- What limits the field of view of a beam-splitter HUD?
- Why fuse gyro and magnetometer data for a heading display?
- How do you compute a waypoint's position on the display?
- What design principles make a HUD readable at a glance?

[PROJECT 03]

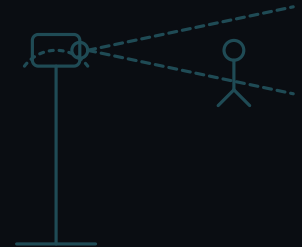
Autonomous Sentry Tower

A tower that spots motion, slews to it, and tracks it across the yard
– a perimeter that watches on its own

\$90

advanced

2 weekends



// why this matters

The modern border and base perimeter is watched by autonomous sensor towers: they detect motion, classify what it is, slew a camera to track it, and alert an operator – no human staring at a screen. Build a small one: a pan-tilt camera head on a mast, a wide-field motion trigger, a person / vehicle classifier, a tracking loop that keeps the target centred, and an alert with a snapshot to your phone (or your command map in project 6). Watch it wake up, turn, and follow you across the yard. It is perception, control, and systems integration – and, because it only watches, entirely a sensor.

WHAT THIS PROVES TO A RECRUITER

You understand layered detection (a cheap trigger plus camera confirmation), object classification on the edge, pan-tilt tracking control (visual servoing), event logging and alerting, and designing an unattended system that runs reliably. That is the core of autonomous-surveillance and perimeter-security engineering.

→ **the job this maps to:** autonomous-sensor and perimeter-security engineering – sentry-tower, security-camera, and base-security systems teams.

! WATCH YOUR OWN GROUND ONLY

This is a sensor tower, never a weapon: nothing gets mounted on it that can harm, ever. Point it only at property you own or control, don't record neighbours, public spaces, or people who haven't consented, and store footage responsibly. A tower that watches your yard is a security camera; a tower aimed at others is surveillance.

// how it works

- ▶ **Cheap trigger, smart confirm:** Running a camera classifier constantly is expensive. A PIR or microwave-radar sensor with a wide field cheaply detects motion, and only then does the camera wake, look, and classify. Layered detection is how real towers stay efficient.
- ▶ **Classification on the edge:** A lightweight neural network (a MobileNet-class model on a Pi, or with a Coral accelerator) distinguishes person, vehicle, and animal – so the tower ignores the neighbour’s cat and alerts on a person.
- ▶ **Tracking (visual servoing):** Once a target is classified, the pan-tilt head steers to keep it centred using the same control loop as a tracking robot – pixel offset in, servo velocity out – so the target stays in frame as it moves.
- ▶ **Alerting and logging:** Each event gets a timestamp, class, snapshot, and track, sent to your phone (or the command map) and logged. A good sentry is a good record keeper.
- ▶ **Running unattended:** Weatherproofing, power (solar plus battery), watchdogs, and failure handling are half the engineering – a sentry that dies at 3 a.m. is useless. Reliability is the discipline.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Pan-tilt head + servos	2-axis bracket, MG996R ×2	1	\$15
Camera + compute	Raspberry Pi 4 / 5 + camera (Coral optional)	1	\$45
Trigger sensor	PIR or 24 GHz microwave motion module	1	\$6
Mast + enclosure	pole, weatherproof box	1	\$14
Power	battery + small solar (optional)	1	\$10

// build steps

1 Build the head

Assemble the pan-tilt bracket with the camera, drive both axes from the Pi, and confirm full sweep and smooth motion.

2 Add the trigger

Mount the PIR / radar sensor with a wide field. Have motion wake the camera and grab a frame; log each trigger.

3 Classify

Run a person / vehicle / animal classifier on the triggered frame. Tune it so a person is reliably detected and animals are ignored (or logged separately).

4 Track

When a person is confirmed, close the loop: compute the pixel offset from centre and drive the pan-tilt to follow. Walk across the yard and watch it track you.

5 Deploy and alert

Mount it on the mast in a weatherproof box, add alerts with snapshots to your phone (or the command map), power it, and let it run overnight. Review the log in the morning.

// interview questions

- Why use a cheap trigger sensor before waking the camera?
- How does an edge classifier distinguish a person from an animal?
- How does the pan-tilt tracking loop keep a target centred?
- What makes an unattended sensor reliable?
- How should a sentry log and communicate events?
- What are the privacy and legal constraints on a surveillance sensor?

[PROJECT 04]

RF Ground Station

A fixed station that watches the whole radio spectrum, decodes what's public, and flags anything new

\$80

intermediate

1-2 weekends



// why this matters

A signals station doesn't hunt one signal; it watches the whole spectrum, all the time. Build a permanent RF ground station: an SDR on a mast antenna, continuously sweeping the bands, logging what is on the air, decoding the public signals – aircraft (ADS-B), ships (AIS), weather satellites, broadcast – and raising an alert when a new emitter appears where there was none. Its waterfall dashboard becomes your radio picture of the area. It is the receive-only heart of a signals post, and it feeds the command map with everything that transmits nearby.

WHAT THIS PROVES TO A RECRUITER

You understand wideband spectrum monitoring, automated sweeping and logging, signal and change detection ("new emitter"), decoding standard public protocols, antenna and station design, and building a reliable always-on receiver. Spectrum awareness is core to EW, SIGINT, and spectrum-management roles.

→ **the job this maps to:** spectrum-monitoring, SIGINT, and electronic-warfare engineering – spectrum-management, defense-RF, and monitoring-system teams.

! PUBLIC SIGNALS ONLY

Receive-only, and decode only signals meant to be public – ADS-B, AIS, weather satellites, broadcast, unencrypted telemetry. Intercepting or decoding private communications (phones, private voice, anything not broadcast to the public) is illegal in most countries. Watch the spectrum; don't listen to people.

// how it works

- ▶ **Sweeping the spectrum:** An SDR retunes across the bands in steps, capturing a power spectrum at each; stitched together it is a wideband survey. Repeated continuously, it becomes a time-lapse of the radio environment.
- ▶ **Baseline and change detection:** Log the average power per frequency over days to build a baseline; a signal where the baseline is empty is a new emitter. Flagging those is the station's alert function – spectrum change detection.
- ▶ **Decoding the public:** Dedicated decoders turn broadcasts into data: ADS-B (aircraft positions), AIS (ships), NOAA / Meteor weather images, FM and DAB. Each becomes a feed on your map.
- ▶ **The station:** A fixed mast antenna (a discone for wideband, plus dedicated antennas), low-loss coax, an LNA, and an SDR into a Pi running always-on software – with a waterfall dashboard you can open from any device.
- ▶ **Feeding the picture:** Every decoded track and every emitter alert is published to the command

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
SDR(s)	RTL-SDR (or two: one sweeping, one decoding)	1-2	\$30
Antennas	discone (wideband) + ADS-B / AIS antennas	—	\$30
LNA + coax	wideband LNA, low-loss coax	—	\$12
Compute	Raspberry Pi 4 (or existing)	1	\$25
Enclosure / mast	weatherproof box, mast	1	\$8

// build steps

1 Stand up the antenna

Mount a discone on a mast with low-loss coax and an LNA at the antenna. Confirm strong reception across VHF / UHF on a waterfall.

2 Sweep and log

Run a continuous spectrum sweep and log power versus frequency versus time. View the survey as a waterfall image over a day.

3 Build the baseline and alerts

Average several days of sweeps into a baseline, then flag frequencies that light up above it — your new-emitter alert. Tune the threshold.

4 Add decoders

Run ADS-B and AIS decoders (and a weather-satellite receiver if you like) and confirm aircraft and ships appear with positions.

5 Publish the picture

Push decoded tracks and alerts to a dashboard or the command map, box the station weatherproof, and let it run for a week. Review what your area's spectrum actually looks like.

// interview questions

- How does an SDR survey a spectrum far wider than it can capture at once?
- How do you detect a new emitter against a noisy baseline?
- Which signals are legal to decode, and which are not?
- Why does antenna and LNA placement dominate station performance?
- How do ADS-B and AIS broadcast position?
- How would you make an always-on receiver station reliable?

[PROJECT 05]

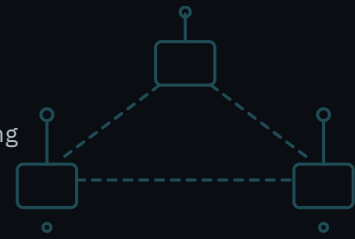
Mesh Field Radio

Pocket radios that form their own network – text and position sharing with no cell towers at all

\$70

intermediate

1 weekend



// why this matters

When the network is down, jammed, or simply absent, teams still need to talk and know where each other are. A LoRa mesh does exactly that: small radios that relay for each other, carrying short encrypted messages and GPS positions across kilometres with no infrastructure. Build a set of nodes, pair them, test range across your town, and watch teammate dots appear on a map – and on your HUD from project 2. It is networking, RF, and embedded in one, and it is the connective tissue of the whole situational-awareness stack.

WHAT THIS PROVES TO A RECRUITER

You understand LoRa modulation and link budget, mesh routing and relaying, position sharing and message encryption, node power management, and real-world range testing. Off-grid mesh networking is core to tactical-comms and resilient-communications engineering.

→ **the job this maps to:** tactical-communications and resilient-network engineering – mesh-radio, off-grid comms, and secure-communications teams.

// how it works

- ▶ **LoRa:** A spread-spectrum modulation that trades data rate for range and sensitivity – kilobits per second over kilometres at milliwatts. Perfect for text and positions, useless for video, which is the point.
- ▶ **Meshing:** Each node relays messages it hears, so a message hops node to node beyond any single radio's range. Flooding with hop limits keeps it simple; smarter routing keeps it efficient.
- ▶ **Position sharing:** Nodes with GPS broadcast their position periodically; every node keeps a table of teammates and their last known location – the data that draws dots on a map or a HUD.
- ▶ **Encryption:** Messages are encrypted with a shared channel key, so the mesh is private to your team even though anyone can receive the raw packets.
- ▶ **Range and power:** Antenna height and gain, terrain, and transmit power set range; sleep modes and duty cycling stretch a battery to days. Range testing is where the link budget becomes real.

BUILD NOTE

Don't write a mesh stack from scratch. Meshtastic-compatible ESP32 LoRa boards with GPS get you a working encrypted mesh in an afternoon; the engineering is in antennas, placement, range testing, and bridging the data into your HUD and command map.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
LoRa + GPS nodes (×3)	ESP32 LoRa boards with GPS (Meshtastic-compatible)	3	\$54
Antennas	tuned for your ISM band	3	\$9
Batteries + cases	LiPo, 3D-printed cases	3	\$7

// build steps

1 Flash and pair

Flash mesh firmware on three nodes, set a private channel key, and confirm they exchange messages on the bench.

2 Add positions

Enable GPS position sharing and open the map view. Confirm each node's dot appears with its last-known position.

3 Test range

Walk or drive nodes apart and log the distance at which the direct link drops. Try antenna height and watch the range change.

4 Test relaying

Place a third node between two out-of-range nodes and confirm messages hop through it – the mesh working as designed.

5 Feed the stack

Bridge one node to your HUD (project 2) and command map (project 6) so teammate positions and messages appear there. Now the team is connected with no infrastructure.

// interview questions

- Why does LoRa achieve such long range at low power?
- How does a mesh network extend range beyond a single radio?
- How do nodes share and track teammate positions?
- How is a mesh channel kept private?
- What determines LoRa range in the real world?
- How would you extend the battery life of a field node?

[PROJECT 06]

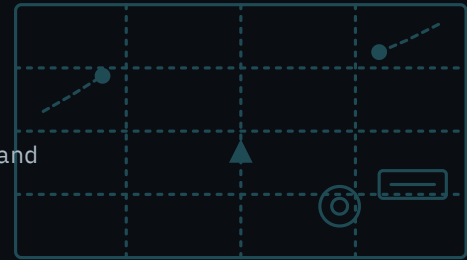
Command Map (Common Operating Picture)

One map that fuses every sensor you built – tracks, alerts, and teammates in a single live picture

\$25

advanced

2 weekends



// why this matters

Every sensor you have built so far is a separate window; command and control is about fusing them into one picture. Build a common operating picture: a server that ingests the sentry tower's detections, the RF station's aircraft and ship tracks and emitter alerts, and the mesh network's teammate positions, and renders them all on a live map with a timeline and an alert feed – the same idea behind the software that runs modern defense operations. It is systems and software engineering: data models, message buses, fusion, and a UI built for decisions. It is the capstone that turns six projects into one system.

WHAT THIS PROVES TO A RECRUITER

You understand systems integration, data models for tracks and events, publish / subscribe messaging, track-level sensor fusion, map-based UI design for decisions, and building a reliable, replayable service. Command-and-control software is one of the fastest-growing areas of defense engineering.

→ **the job this maps to:** command-and-control, systems, and defense-software engineering – mission-software, sensor-fusion, and integration teams.

! YOUR SENSORS, YOUR PICTURE

This fuses data from your own sensors on your own property and your own team. Don't aggregate other people's data into it, and secure the server – it is a live map of your sensors and your people.

// how it works

- ▶ **The data model:** Everything becomes a track (an object with position, class, and history) or an event (a detection, an alert), each with a timestamp and a source. Defining that model well is most of the work.
- ▶ **The message bus:** Sensors publish to a broker (MQTT is the classic choice); the map subscribes to everything. Publish / subscribe decouples sensors from the display, so you can add a sensor without touching the map.
- ▶ **Fusion:** When two sensors see the same thing (the sentry's person and the mesh's teammate at that spot), the map correlates them into one track instead of two – track-level fusion, which is where the picture becomes trustworthy.
- ▶ **The map UI:** A web map draws tracks, alerts, sensor footprints, and a timeline, with filters – designed for quick decisions, not for looking pretty. Alerts float to the top.
- ▶ **Reliability and history:** A database stores every track and event so you can replay a night; the service restarts itself, buffers when sensors drop, and never loses an alert. That is what makes it operational.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Server	Raspberry Pi 4 (or your laptop)	1	\$25
Software	MQTT broker, Python or Node backend, web map	–	free
Sensors	projects 3, 4, and 5	–	built

// build steps

1 Define the model

Write the track and event schemas (position, class, source, time, history). Keep them simple and shared by every sensor.

2 Stand up the bus

Run an MQTT broker and have each sensor (sentry, RF station, mesh bridge) publish tracks and events to it. Watch messages flow.

3 Build the map

Create the web map that subscribes and draws tracks, alerts, and sensor positions live. Add a timeline and an alert feed.

4 Fuse tracks

Implement correlation so the same object seen by two sensors becomes one track and confidence rises. Test by walking past the sentry with a mesh node in your pocket.

5 Store, replay, harden

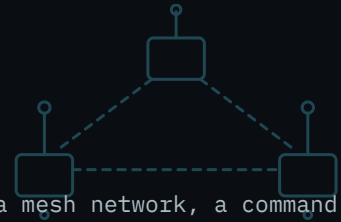
Add a database, a replay mode, auto-restart, and buffering. Run the full stack overnight and replay it in the morning – your own operating picture.

// interview questions

- What is a common operating picture, and why does it matter?
- Why use publish / subscribe messaging between sensors and a display?
- How would you model tracks and events from different sensors?
- What is track-level fusion, and why is it hard?
- What makes a map UI effective for decisions?
- How do you make a C2 service reliable and replayable?

[WRAP]

Situational awareness is a stack – sense, connect, display, decide.



A night-vision monocular, a HUD goggle, a sentry tower, an RF station, a mesh network, a command map – six builds that together are a personal situational-awareness stack, the same architecture behind the systems modern defense runs on. Every one senses, connects, or displays; none of them harm. The value is in the integration: optics, edge inference, control loops, spectrum monitoring, networking, and systems software, working as one. That is what the defense-tech companies everyone is talking about actually hire for.

// the skill ladder

Build them in order and each one hands you the technique the next one needs:

imaging → HUD optics → autonomous sensing → spectrum monitoring → mesh networking → command & control

! THE LINE THIS GUIDE STAYS ON

Every build here is **receive-only, a sensor, a network, or a display**. Nothing targets, jams, intercepts private communications, or carries anything that can harm – and nothing ever gets mounted on the sentry tower but a camera. Watch your own ground, decode only public signals, respect people who haven't consented, and follow local law. Build the picture, not the weapon.

DO THIS

Build even one. Film it – a pitch-black room appearing on screen, a waypoint sliding across your view as you turn your head, a tower slewing to follow you, a new emitter lighting up a waterfall, a teammate's dot appearing with no cell signal, a map fusing it all – and put the code on GitHub with your architecture, your data model, and an honest note on scope and privacy. One working node and one clean repo prove you can build the stack.

Full build notes, parts links, and the next field guide go out in **The Build List**. The people actually shipping these – and trading combiner and antenna tricks – are in **Ohm's Club**.

– Enes (@1nska) · Electrical Engineer